LAB IMPLEMENTATION EXPERIMENT 7

Experiment 7: Text Generation using Recurrent Neural Networks (RNNs) and LSTMs

Generative AI Laboratory (CO2)

Date: 22 October 2026

Aim

To build character-level text generators using a Recurrent Neural Network (RNN) and a Long Short-Term Memory (LSTM) network, to train them on Shakespeare's plays, and to generate new text with controllable randomness.

Theory

A **recurrent neural network** processes a sequence $x_1, \ldots, x_T$ while carrying a hidden state $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$. A **language model** estimates $p(x_t \mid x_{<t})$ and is trained with cross-entropy on next-token prediction; its quality is measured by *perplexity* $= \exp(\text{cross-entropy})$.

Plain RNNs suffer from vanishing/exploding gradients over long sequences. The **LSTM** adds a cell state c_t and gates:

$$f_t = \sigma(W_f[h_{t-1}, x_t]),\ i_t = \sigma(W_i[h_{t-1}, x_t]),\ o_t = \sigma(W_o[h_{t-1}, x_t]),$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_c[h_{t-1}, x_t]), \qquad h_t = o_t \odot \tanh(c_t),$$

so information can be remembered or forgotten selectively.

To generate text, the softmax distribution of the next character is sampled repeatedly and the sampled character is fed back. A *temperature* T rescales the logits, $p_i \propto \exp(z_i/T)$: $T < 1$ makes the output conservative, $T > 1$ more random.

Steps

1. Load the Shakespeare corpus (about 1.1 million characters), build the character vocabulary and integer encoding.
2. Create overlapping training windows of 100 characters (stride 50) with targets shifted by one character; split into train/validation.
3. Define two models: Embedding → SimpleRNN(256) → Dense, and Embedding → LSTM(256) → Dense.
4. Train both with Adam and gradient clipping; plot loss and perplexity.
5. Write a sampling function with temperature and generate text from a seed.
6. Compare the models and the temperatures using the share of generated words that are valid.

Code

```python
import os, time
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
import numpy as np, matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
keras.utils.set_random_seed(11)
path = keras.utils.get_file("shakespeare.txt", "https://storage.googleapis.com/download.tensorflow.org/data/shakespeare.txt")
text = open(path, encoding="utf-8").read()
chars = sorted(set(text)); V = len(chars)
c2i = {c: i for i, c in enumerate(chars)}; i2c = np.array(chars)
data = np.array([c2i[c] for c in text], dtype=np.int32)
print("characters used:", len(text), "| vocabulary size:", V)
print(text[:200])

SEQ, BATCH, STRIDE = 100, 128, 50
starts = np.arange(0, len(data) - SEQ - 1, STRIDE)                  # overlapping windows
X = np.stack([data[i:i + SEQ] for i in starts]); Y = np.stack([data[i + 1:i + SEQ + 1] for i in starts])
perm = np.random.RandomState(0).permutation(len(X)); n_val = len(X) // 20
val_ds = tf.data.Dataset.from_tensor_slices((X[perm[:n_val]], Y[perm[:n_val]])).batch(BATCH)
train_ds = (tf.data.Dataset.from_tensor_slices((X[perm[n_val:]], Y[perm[n_val:]])).shuffle(5000, seed=0)
            .batch(BATCH, drop_remainder=True).prefetch(2))
print("windows:", len(X), "| train batches:", len(train_ds), "| val windows:", n_val)
print("input :", repr("".join(i2c[X[0][:50]])))
print("target:", repr("".join(i2c[Y[0][:50]])))

def build(kind, units=256):
    Rnn = {"SimpleRNN": layers.SimpleRNN, "LSTM": layers.LSTM}[kind]
    return keras.Sequential([
        layers.Input((None,)),
        layers.Embedding(V, 64),
        Rnn(units, return_sequences=True),
        layers.Dense(V),
    ], name=kind.lower() + "_lm")
for k in ("SimpleRNN", "LSTM"):
    m = build(k); print(f"{k}: {m.count_params():,} parameters")
build("LSTM").summary(print_fn=print)

loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)
hist, models = {}, {}
for kind, epochs in (("SimpleRNN", 6), ("LSTM", 12)):
    m = build(kind); m.compile(keras.optimizers.Adam(3e-3, clipnorm=1.0), loss)
    t0 = time.time()
    hist[kind] = m.fit(train_ds, validation_data=val_ds, epochs=epochs, verbose=2).history
    models[kind] = m
    print(f"{kind}: {time.time() - t0:.0f}s | final val loss {hist[kind]['val_loss'][-1]:.3f} | "
          f"val perplexity {np.exp(hist[kind]['val_loss'][-1]):.2f}\n")
```

```
48
49 fig, ax = plt.subplots(1, 2, figsize=(10, 3.4))
50 for k, c in (("SimpleRNN", "tab:orange"), ("LSTM", "tab:blue")):
51     e = range(1, len(hist[k]["loss"]) + 1)
52     ax[0].plot(e, hist[k]["loss"], "o-", c=c, label=k + " train"); ax[0].plot(e,
       hist[k]["val_loss"], "s--", c=c, label=k + " val")
53     ax[1].plot(e, np.exp(hist[k]["val_loss"]), "s-", c=c, label=k)
54 ax[0].set(title="Cross-entropy loss", xlabel="epoch"); ax[0].legend(fontsize=7)
55 ax[1].set(title="Validation perplexity (lower is better)", xlabel="epoch"); ax[1].
       legend()
56 plt.tight_layout(); plt.show()
57
58 def generate(model, seed, n=400, temperature=1.0):
59     ids = [c2i[c] for c in seed]
60     out = list(seed)
61     for _ in range(n):
62         logits = model(np.array([ids[-SEQ:]]), training=False)[0, -1].numpy() /
       temperature
63         p = np.exp(logits - logits.max()); p /= p.sum()
64         nxt = np.random.choice(V, p=p)
65         ids.append(nxt); out.append(i2c[nxt])
66     return "".join(out)
67 np.random.seed(5)
68 print("generate() defined; sampling with temperature scaling")
69
70 seed = "ROMEO:\n"
71 for T in (0.3, 0.7, 1.0):
72     print(f"======== LSTM, temperature {T} ========")
73     print(generate(models["LSTM"], seed, 350, T), "\n")
74
75 np.random.seed(5)
76 for k in ("SimpleRNN", "LSTM"):
77     print(f"======== {k} ========")
78     print(generate(models[k], "First Citizen:\n", 300, 0.7), "\n")
79
80 vocab_words = set(text.lower().replace("\n", " ").split())
81 def real_word_rate(s):
82     w = [x.strip(".,;:!?'\"") for x in s.lower().replace("\n", " ").split()]
83     w = [x for x in w if x]
84     return sum(x in vocab_words for x in w) / max(1, len(w))
85 rows = []
86 np.random.seed(1)
87 for k in ("SimpleRNN", "LSTM"):
88     for T in (0.3, 0.7, 1.0, 1.3):
89         s = generate(models[k], "ROMEO:\n", 600, T)
90         rows.append((k, T, real_word_rate(s)))
91 print(f"{'model':10s} {'temp':>5s}  words found in training vocabulary")
92 for k, T, r in rows: print(f"{k:10s} {T:5.1f}  {100 * r:5.1f}%")
93 fig, ax = plt.subplots(figsize=(6, 3))
94 for k, c in (("SimpleRNN", "tab:orange"), ("LSTM", "tab:blue")):
95     ax.plot([r[1] for r in rows if r[0] == k], [100 * r[2] for r in rows if r[0] ==
       k], "o-", c=c, label=k)
96 ax.set(xlabel="temperature", ylabel="% valid words", title="Valid-word rate vs
       temperature"); ax.legend()
97 plt.tight_layout(); plt.show()
```

Listing 1. `experiment_code.py`

Output

Imports and data

```
characters used: 1115394 | vocabulary size: 65
First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You are all resolved rather to die than to famish?

All:
Resolved. resolved.

First Citizen:
First, you
```

Building the tf.data pipeline of (input, target) windows

```
windows: 22306 | train batches: 165 | val windows: 1115
input : 'First Citizen:\nBefore we proceed any further, hear'
target: 'irst Citizen:\nBefore we proceed any further, hear '
```

Defining the SimpleRNN and LSTM language models

```
SimpleRNN: 103,041 parameters
LSTM: 349,569 parameters
Model: "lstm_lm"
+---------------------------------+------------------------+---------------+
| Layer (type)                    | Output Shape           |       Param # |
+---------------------------------+------------------------+---------------+
| embedding_2 (Embedding)         | (None, None, 64)       |         4,160 |
+---------------------------------+------------------------+---------------+
| lstm_1 (LSTM)                   | (None, None, 256)      |       328,704 |
+---------------------------------+------------------------+---------------+
| dense_2 (Dense)                 | (None, None, 65)       |        16,705 |
+---------------------------------+------------------------+---------------+
 Total params: 349,569 (1.33 MB)
 Trainable params: 349,569 (1.33 MB)
 Non-trainable params: 0 (0.00 B)
```

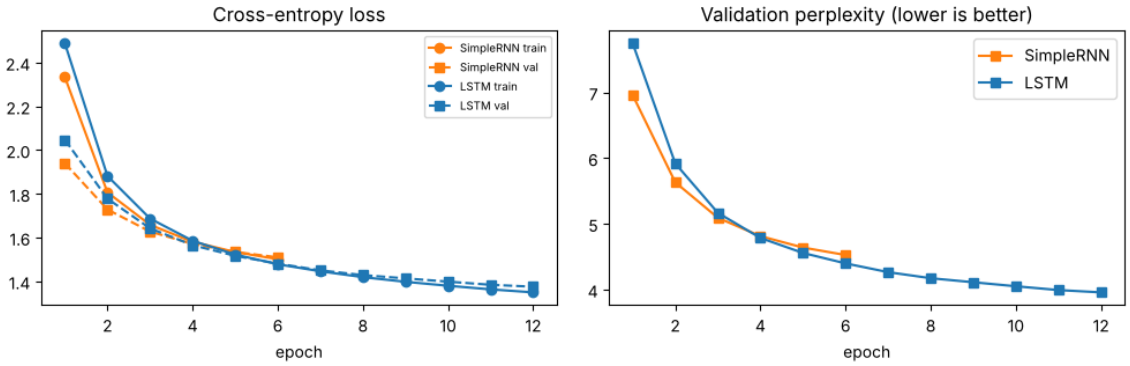
Training both models (cross-entropy / perplexity per epoch)

```
Epoch 1/6
165/165 - 14s - 86ms/step - loss: 2.3337 - val_loss: 1.9400
Epoch 2/6
165/165 - 14s - 86ms/step - loss: 1.8067 - val_loss: 1.7286
Epoch 3/6
165/165 - 13s - 80ms/step - loss: 1.6585 - val_loss: 1.6276
Epoch 4/6
165/165 - 17s - 101ms/step - loss: 1.5810 - val_loss: 1.5713
Epoch 5/6
165/165 - 20s - 120ms/step - loss: 1.5340 - val_loss: 1.5346
Epoch 6/6
165/165 - 20s - 122ms/step - loss: 1.5012 - val_loss: 1.5097
```

```
SimpleRNN: 98s | final val loss 1.510 | val perplexity 4.53

Epoch 1/12
165/165 - 48s - 289ms/step - loss: 2.4883 - val_loss: 2.0473
Epoch 2/12
165/165 - 50s - 306ms/step - loss: 1.8817 - val_loss: 1.7782
Epoch 3/12
165/165 - 39s - 234ms/step - loss: 1.6872 - val_loss: 1.6424
Epoch 4/12
165/165 - 52s - 314ms/step - loss: 1.5855 - val_loss: 1.5655
Epoch 5/12
165/165 - 36s - 215ms/step - loss: 1.5227 - val_loss: 1.5167
Epoch 6/12
165/165 - 35s - 213ms/step - loss: 1.4782 - val_loss: 1.4811
Epoch 7/12
165/165 - 35s - 212ms/step - loss: 1.4461 - val_loss: 1.4503
Epoch 8/12
165/165 - 40s - 244ms/step - loss: 1.4192 - val_loss: 1.4285
Epoch 9/12
165/165 - 51s - 306ms/step - loss: 1.3980 - val_loss: 1.4137
Epoch 10/12
165/165 - 35s - 211ms/step - loss: 1.3799 - val_loss: 1.3987
Epoch 11/12
165/165 - 34s - 207ms/step - loss: 1.3634 - val_loss: 1.3843
Epoch 12/12
165/165 - 35s - 215ms/step - loss: 1.3496 - val_loss: 1.3755
LSTM: 489s | final val loss 1.375 | val perplexity 3.96
```

Training curves and perplexity



Text generation function

```
generate() defined; sampling with temperature scaling
```

Text generated by the LSTM at different temperatures

```
======== LSTM, temperature 0.3 ========
ROMEO:
I have seen the man that a word will not stocks thee
And the sun and the gods of his hearts
And take the season of this place of the hands
And think to the company to me to the people,
To see the sea than the fire of words.

CORIOLANUS:
Why, then I say the gates of my worship,
```

```
And seem the child of that made to the people,
Which the prince that hav

======== LSTM, temperature 0.7 ========
ROMEO:
I have sent the man hath thousand to know.

Lord Rastio:
Be, give me here to hear me, and make you are ready
Than the countress of me a soul too thee,
Which some which command as he hath made the sun
Which hanged him or to the shapes, let thee and
But or one than the chopper of a father's by
As I will tell in repersed, that it is
report of good liv

======== LSTM, temperature 1.0 ========
ROMEO:
I am wailt a God, make my blood upon this boot,
I knew it in pird,
Dakes him now we once, 'tis dreame?
To my viewan, lamentation.

LUCIO:
Why, what I now, pass! he is. What seem'd sheph
some father, say to our lurder of in all,
And in the morns to make thee changing.

CAPULET:
A meaning to live to the Edward,
Lived man we shall pitius dead of porit
```

Comparison - SimpleRNN vs LSTM (temperature 0.7)

```
======== SimpleRNN ========
First Citizen:
His like a slave,
The head woo my braves to Fither made you prove me the grace! who didst them much you, sir; but
    thou sings and the galling the worshy mele mean'd with thy heart!

DUKE VINCENTIO:
He had for the mind, that they arance to-morrow he heart, when they be putsenger'd the counten
    are desi

======== LSTM ========
First Citizen:
You may canst the company; and therefore stay your father,
Or slain hath strew your prince to supper them
Hath determined with him: boy know I have gates,
And laid the countress of me a soul!
What entreat a strange?

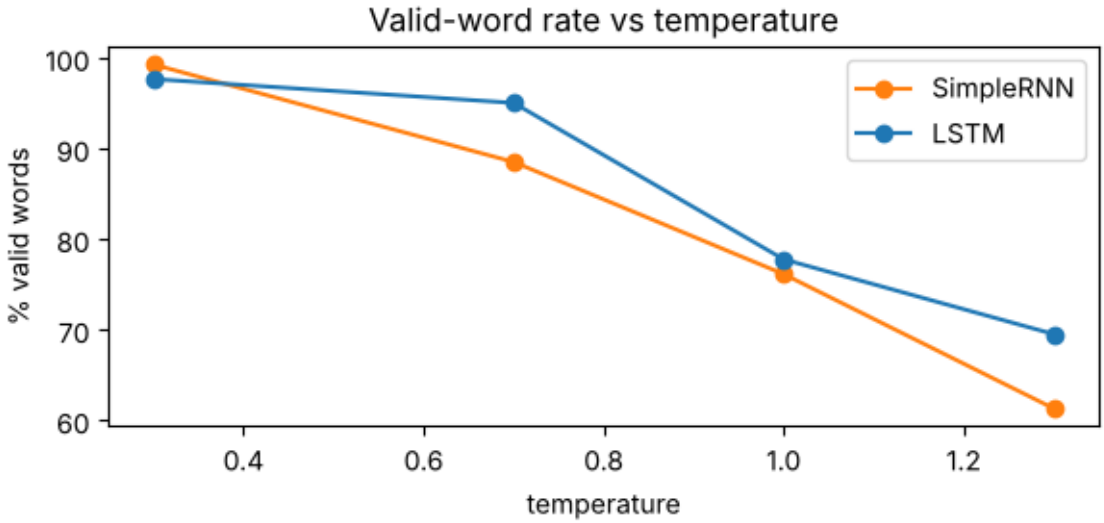
SICINIUS:
We have send me seendly.

PARIS:
Poor God, then thou wert passay heart
As
```

Quality metrics of generated text

```
model       temp  words found in training vocabulary
```

```
SimpleRNN    0.3   99.3%
SimpleRNN    0.7   88.5%
SimpleRNN    1.0   76.1%
SimpleRNN    1.3   61.3%
LSTM         0.3   97.7%
LSTM         0.7   95.0%
LSTM         1.0   77.8%
LSTM         1.3   69.5%
```



Conclusion

Both recurrent models learned the format of the plays (speaker names in capitals followed by a colon, line breaks, capitalisation) and mostly real English words from raw characters. The LSTM (12 epochs, 489 s) reached a validation loss of 1.375, i.e. a perplexity of 3.96, compared with 1.510 (perplexity 4.53) for the SimpleRNN (6 epochs, 98 s), so the gated memory gives a clearly better model, although the LSTM also trained for more epochs and took about five times longer. Generated text is locally fluent but has little global meaning, as expected from a character-level model. Temperature controls the trade-off: at $T = 0.7$ 95.0% of the LSTM's words were valid vocabulary words, at $T = 1.0$ 77.8% and at $T = 1.3$ only 69.5%, while low temperatures give safe but repetitive text. (At $T = 0.3$ the SimpleRNN scored slightly higher, 99.3% versus 97.7%, because its output is more repetitive.)